

R Testing Frameworks

In this seventh article in the 'R, Statistics and Machine Learning' series, we shall explore the various testing frameworks available in R, and how to use them.



We will be using R version 4.1.0 installed on Parabola GNU/Linux-libre (x86-64) for the code snippets.

RUnit

RUnit is a unit testing framework for R written by Thomas König, Klaus Jünemann, and Matthias Burger and is similar to the xUnit testing tools. The package is released under the GPLv2 license, and the last major version is 0.4.32. It is not actively developed, but a number of R packages use the RUnit framework and hence the package is maintained. You can install the package in an R console as shown below:

```
> install.packages("RUnit")
...
*** installing help indices
** building package indices
** installing vignettes
** testing if installed package can be loaded from
temporary location
** testing if installed package can be loaded from final
location
** testing if installed package keeps a record of temporary
installation path
* DONE (RUnit)
```

You can load the library to the R session using the following command:

```
> library(RUnit)
```

Consider a square function:

```
> square <- function(x) return (x * x)
```

A *test.square* function can be written to use the *checkEquals* function to assert expected values, as shown below:

```
> test.square <- function() {
  checkEquals(square(2), 4)
}
```

The test function can now be executed in the R console, as illustrated below:

```
> test.square()
[1] TRUE
```

You can now increase the scope of the tests for zero, negative numbers, fractions and complex numbers as well.

```
> checkEquals(square(0), 0)
[1] TRUE
> checkEquals(square(-1), 1)
[1] TRUE
> checkEquals(square(1/2), 1/4)
[1] TRUE
> checkEquals(square(1i), -1+0i)
[1] TRUE
```

The special constants in R can also be tested with the *checkEquals()* function as follows:

```
> checkEquals(NA, NA)
[1] TRUE
> checkEquals(Inf, Inf)
[1] TRUE
> checkEquals(NaN, NaN)
Error in mode(current) : object 'Nan' not found
> checkEquals(NaN, NaN)
[1] TRUE
> checkEquals(NULL, NULL)
[1] TRUE
```

The 'i' for complex numbers in R must be prefixed with a number; otherwise, it will throw an error. This error can be caught using the *checkException* function, as shown below:

```
> checkException(square(i), -1+0i)
```